TMF8829 Host driver communication

Application Note

Published by ams-OSRAM AG
Tobelbader Strasse 30,
8141 Premstaetten Austria
Phone +43 3136 500-0
ams-osram.com

TMF8829 Host driver communication

Application Note No. AN001096

Valid for:
TMF8829

Abstract

This document specifies the communication between TMF8829 and a host. It describes the sensor start sequence, running measurements, retrieving result or histogram data and how to interpret this sensor data. In addition, it shows procedures to increase measurement accuracy with oscillator trimming and oscillator drift correction. Information for motion detection and 3D point cloud distance correction is given.

Table of contents

1 Introduction

This document describes the communication between the TMF8829 device and a host. This document provides information about the TMF8829 device startup, the bootloader mode and the application. Information with examples for the appropriate configuration, performing measurements and result evaluation is provided in detail.

1.1 Communication interfaces

The TMF8829 provides a I²C, I3C and a SPI communication interface. Please refer to the datasheet for a detailed description of the interfaces.

2 Startup and shutdown

This chapter describes the startup behavior of the TMF8829 device and how the host should communicate with the TMF8829 during and immediately after the startup.

The device will be started in bootloader mode. In this mode bootloader commands are available and a RAM patch download shall be performed. The running application is exposed in the register TMF8829_APP_ID 0x00. The following sequence describes a startup and a shutdown:

Table 1: Startup sequence and shutdown sequence

Function	Address	Data	Register name	Comment
Enable				
EN = High	…	…	…	Enable pin to high
Wait				3ms … enable wait time
Power up				
Write	0xF8	0x04	ENABLE	pon = 1; powerup_select = 0
Wait				3ms(1) … power up wait time
Read	0xF8	0x84	ENABLE	cpu_ready = 1
Read	0x00	0x80	TMF8829_APP_ID	app_id = bootloader application ID
Switch off unused communication interface				
Write	0x08	0x20	TMF8829_CMD_STAT	Disable SPI interface
Wait				1ms(2) … BL cmd execution wait time
Read	0x08	0x00		BL_STAT_SUCCESS
Or				
Write	0x08	0x22	TMF8829_CMD_STAT	Disable I²C/I3C interface
Wait				1ms(2)… BL cmd execution wait time
Read	0x08	0x00		BL_STAT_SUCCESS
Patch download, see chapter RAM patch download				
Read	0x00	0x01	TMF8829_APP_ID	app_id = ROM application id
Enable Register with powerup_select attribute to RAM(3)				
Read	0xF8	0x84	ENABLE	
Write	0xF8	0xA4	ENABLE	powerup_select = 2
Application running				
Power down				
Write	0xF8	0x08	ENABLE	poff = 1
Wait				3ms(4) … disable wait time
Read	0xF8	<=0x7F	ENABLE	cpu_ready=0
Disable				
EN = Low	…	…	…	Enable pin to low

(1) Maximum wait time. TMF8829_APP_ID could be polled for the application id to save waiting time.
(2) Maximum wait time. CMD_STAT could be polled for BL_STAT_SUCCESS to save waiting time.
(3) Set the power up option to RAM. This is required for the Power Modes standby (Sleep mode) and standby timed.
(4) Maximum wait time. The ENABLE register could be polled for cpu_ready=0 to save wait time.

2.1 Standby mode

To put the device into standby state the following sequence must be executed:

Table 2: Device to standby

Function	Address	Data	Register name	Comment
Read	0xF8	Val1	ENABLE	Read ENABLE register
Write	0xF8	0x08 | Val1	ENABLE	poff = 1
Wait				3ms(1) … enable wait time
Read	0xF8		ENABLE	cpu_ready = 0

(1) Maximum wait time. ENABLE register could be polled for right cpu_ready value to save waiting time.

2.2 Wake up device

To wake up the TMF8829 device after it has entered the standby state the following sequence must be executed:

Table 3: Wake up device

Function	Address	Data	Register name	Comment
Read	0xF8	Val1	ENABLE	Read ENABLE register
Write	0xF8	0x24 | Val1	ENABLE	pon = 1, powerup_select = 2
Wait				3ms(1) … enable wait time
Read	0xF8		ENABLE	cpu_ready = 1

(1) Maximum wait time. ENABLE register could be polled for right cpu_ready value to save waiting time.

The wakeup could also be done from standby timed mode.

2.3 Soft reset

A soft reset could be performed with the RESET register. The attribute powerup_select in the ENABLE register should be set to FORCE_BOOTMONITOR 0x01.

Table 4: Soft reset to bootloader or application

Function	Address	Data	Register name	Comment
Reset to bootloader				
Write	0xF8	0x14	ENABLE	powerup_select = 1 (FORCE_BOOTMONITOR)
Wait				3ms wait time
Write	0xF7	0x40	RESET	soft_reset = 1
Wait				3ms(1) … reset wait time
Read	0xF8	0x01	ENABLE	cpu_ready = 1
Read	0x00	0x80	TMF8829_APP_ID	app_id = bootloader application id

(1) Maximum wait time. ENABLE register could be polled for right cpu_ready value to save waiting time.

3 Bootloader protocol

The bootloader / boot-monitor supports a protocol to:

- Switch off the unused communication interface.
- Download a program to the internal RAM.
- Perform ROM/RAM remap and reset to start the application firmware loaded to RAM.

The bootloader ID in the TMF8829_APP_ID register 0x00 is 0x80.

3.1 Command overview

The bootloader protocol is implemented through a register map. The commands are written to register CMD_STAT 0x08. Commands which have a response code are also read from this register. The following commands are supported by the bootloader:

Table 5: Boot-monitor commands

Code (Decimal)	Symbol	Function
22	BL_CMD_START_RAM_APP	Start the RAM application
32	BL_CMD_SPI_OFF	Disable SPI interface
34	BL_CMD_I2C_OFF	Disable I²C/I3C interface
69	BL_CMD_W_FIFO_BOTH	Setup size for FIFO writes to both CPU RAM's in parallel
All other not specified values shall not to be used and the behavior if one of them is set, is undefined.		

The following tables show the possible responses:

Table 6: Boot-monitor status codes

Code (Decimal)	Symbol	Function
0	BL_STAT_SUCCESS	Command executed successfully
1	BL_STAT_ERR_PARAM	One of the parameters is wrong
2	BL_STAT_ERR_ADDR	RAM address out of range
3	BL_STAT_ERR_SIZE	Size of command does not match
4	BL_STAT_ERR_FIFO	FIFO transfer not completed

3.2 Command list

3.2.1 BL_CMD_START_RAM_APP

This command will start the RAM application and is used after a RAM patch download. This command has no parameter.

Table 7: BL_CMD_START_RAM_APP

Address	Register name	Value/range	Meaning
0x08	CMD_STAT	22	Start the RAM application

A successful execution could be checked with a reading of the application ID register 0x00. This register value should show the application ID instead of the bootloader ID.

3.2.2 BL_CMD_SPI_OFF

This command shall switch the MOSI/GPIO0, CSN/GPIO1, SCLK/GPIO2 and MISO/GPIO3 in gpio mode (SPI is off) and switch off the SPI interface in the register INTERFACE. This command has no parameter.

Table 8: BL_CMD_SPI_OFF

Address	Register name	Value/range	Meaning
0x08	CMD_STAT	32	Disable SPI Interface

A successful execution should read the value 0 back from registers CMD_STAT.

3.2.3 BL_CMD_I2C_OFF

This command shall switch the SCL, SDA in gpio mode (I²C/I3C is off) and switch off the I²C/I3C interface in the register INTERFACE. This command has no parameter.

Table 9: BL_CMD_I2C_OFF

Address	Register name	Value/range	Meaning
0x08	CMD_STAT	34	Disable I²C/I3C Interface

A successful execution should read the value 0 back from registers CMD_STAT.

3.2.4 BL_CMD_W_FIFO_BOTH

This command sets the RAM address and size for a FIFO upload. The size parameter is in 32-bit words. The command is used for the RAM patch download.

Table 10: BL_CMD_W_FIFO_BOTH

Address	Register name	Value/range	Meaning
0x08	CMD_STAT	34	Setup FIFO upload
0x09	PAYLOAD	6	Size of command payload
0x0A	ADDRESS0	0..0xff	Address LSB
0x0B	ADDRESS1	0..0xff	
0x0C	ADDRESS2	0..0xff	
0x0D	ADDRESS3	0..0xff	Address MSB
0x0E	WORD_SIZE0	0..0xff	Word size LSB
0x0F	WORD_SIZE1	0..0xff	Word size MSB

A successful execution should read the value 0 back from registers CMD_STAT. There can be two errors that occur. The error BL_STAT_ERR_ADDR if the address pointer points not to RAM or the error BL_STAT_ERR_PARAM if the word size parameter is larger than 0x1000.

3.3 RAM patch download

To perform a RAM patch download the device must be in bootloader mode. The download is based on a FIFO upload and done in the following steps:

1. Setup the FIFO upload with the command BL_CMD_W_FIFO_BOTH.
2. Write the RAM patch to the FIFO register 0xFF. This could be done in one writing or in chunks.
3. Start the RAM application with the command BL_CMD_START_RAM_APP.
4. Read and verify the application ID.

Example with an image size of 0x3000 bytes (word size 0xC00) download in chunks:

Table 11: RAM patch download example

Function	Address	Data	Register	Comment
Write	0x08	0x34	CMD_STAT	Setup FIFO upload
	0x09	0x06	PAYLOAD	Size of command payload
	0x0A	0x00	ADDRESS0	Address LSB
	0x0B	0x00	ADDRESS1	
	0x0C	0x01	ADDRESS2	
	0x0D	0x00	ADDRESS3	Address MSB
	0x0E	0x0C	WORD_SIZE0	Word Size LSB
	0x0F	0x00	WORD_SIZE1	Word Size MSB
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	BL_STAT_SUCCESS expected
Write	0xFF	<Byte_0> … <Byte_99>	FIFO	Write the first 100 Bytes of RAM patch
Write	0xFF	<Byte_100> … <Byte_199>	FIFO	Write the next 100 Bytes of RAM patch
…	…	…	…	…
Write	0xFF	<Byte_(n-100)> … <Byte_ n>	FIFO	Write last Bytes n of RAM patch
Write	0x08	0x16	CMD_STAT	Start the RAM application
Wait				3ms(1)
Read	0x08	0x00	CMD_STAT	BL_STAT_SUCCESS expected
Read	0x00	0x01	TMF8829_APP_ID	0x01 expected for RAM application ID
	0x01		TMF8829_MAJOR	Major application revision
	0x02		TMF8829_MINOR	Minor application revision

(1) Maximum wait time. CMD_STAT could be polled for a BL_STAT_SUCCESS to save waiting time.

4 Application

The application is available after a successful startup as described in chapter “Startup and shutdown”. The application provides registers about the application revision, serial number of the device and access to perform commands. A detailed description of the application registers is available in the datasheet. The typical usage of the application is to configure the device, start a measurement, read out results and stop the measurement.

4.1 Command execution

Host commands are written to register TMF8829_CMD_STAT 0x08. The description of this register is available in the datasheet and provides information about all the available host commands and the status of command execution. It is recommended to do the execution in the following steps:

1. Send the command to the register TMF8829_CMD_STAT 0x08.
2. Read the register TMF8829_CMD_STAT 0x08. The register should have the status STAT_OK or STAT_ACCEPTED and in case of an error a status STAT_ERR_<xxx>. There could be a delay between writing the command and getting the status. Therefore, it is recommended to repeat the reading until the expected result occurs, or a timeout happens.

4.2 Interrupts

The application supports four types of interrupts:

- int0 – Signal that a result frame is available.
- int1 – Signal that a motion was detected.
- int2 – Signal that a proximity object was detected.
- int3 – Signal that a histogram frame is available.

Bit 0 in the register INT_STATUS will be set by the firmware if a result frame is available.

Bit 1 in the register INT_STATUS will be set by the firmware if a motion was detected.

Bit 2 in the register INT_STATUS will be set by the firmware for a proximity detection.

Bit 3 in the register INT_STATUS will be set by the firmware if a histogram frame is available.

The host writes a ‘1’ to a bit in the INT_STATUS Register to clear the corresponding flag.

4.3 Device configuration

The application is configured via a configuration page. The page contains a set of registers that can be read, modified and written to alter an existing configuration. For example, the result format configuration is done in register TMF8829_CFG_RESULT_FORMAT. The configuration page is described in the datasheet. The following steps are required to change the configuration.

1. Load the configuration page and preconfigure the selected mode e.g. CMD_LOAD_CFG_16X16.
2. Change the desired configuration parameters.
3. Store the configuration page and execute the command CMD_WRITE_PAGE.

The TMF8829 device has the option to use different modes. Guaranteed performance is achieved only by using these pre-sets. Convenience functions are provided which are changing the right configuration parameters and described in the datasheet see register TMF8829_CMD_STAT. The following example will change the configuration to focal plane mode 16x16 and a long-distance range with additional changes of the measurement period and publishing histograms. Afterwards, the start of the measurement is performed.

Table 12: Pre-configuration example

Function	Address	Data	Register name	Comment
Write	0x08	0x43	CMD_STAT	CMD_LOAD_CFG_16X16
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK
Write	0x22	0x64	TMF8829_CFG_PERIOD_MS_LSB	periodLSB
		0x00	TMF8829_CFG_PERIOD_MS_MSB	periodMSB
Write	0x2B	0x01	TMF8829_CFG_DUMP_HISTOGRAMS	Optional: Provide raw histograms
Write	0x08	0x14	CMD_STAT	CMD_WRITE_PAGE_AND_ MEASURE
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK

(1) Maximum wait time. CMD_STAT could be polled for a STAT_OK to optimize speed. See chapter Command execution.

Attention:

The configuration must be chosen so that the result frame will not exceed 8192 bytes. The configuration error STAT_ERR_CONFIG will occur for this wrong setting.

4.4 Start / stop a measurement

To start a measurement, execute the command CMD_MEASURE. To stop the measurement, execute the command CMD_STOP. After issuing the stop command, the application will terminate measurements as quickly as possible. This time depends on the internal sensor state.

Table 13: Start / stop a measurement

Function	Address	Data	Register name	Comment
Write	0x08	0x10	CMD_STAT	CMD_MEASURE
Wait				1ms(1)
Read	0x08	0x01	CMD_STAT	STAT_ACCEPTED
Measurement	…	…	…	…
Note: If device_sleep option in register TMF8829_CFG_POWER_MODES is set, perform a wake up of the device first.(2)				
Write	0x08	0xFF	CMD_STAT	CMD_STOP
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK

(1) Maximum wait time. CMD_STAT could be polled for a STAT_ACCEPTED or STAT_OK.
(2) The measurement could not be stopped if the device is in standby timed mode. To optimize speed, the cpu_ready bit in the ENABLE register could be read. If the CPU is ready, a wake up is not required.

4.5 Read frames

An interrupt is triggered as soon as a frame is available and the host can readout the frame. The readout should start at register FIFOSTATUS 0xFA. The firmware acts in a non-blocking measurement mode for measurements without histogram dumping. If the host is not fast enough with reading of data, frames could be lost. As soon as a single byte of the FIFO register is read, the firmware will not overwrite the FIFO with new data. For slow hosts, the measurement period could be adjusted in the configuration settings to not lose frames. If histogram mode is enabled, data will not be overwritten by newer result / histogram data. The firmware acts in a blocking measurement mode.

Attention:

If the measurement is done with power mode standby timed, the register INT_STATUS can only be read if the cpu is ready and not in standby timed. This must be considered if the interrupt handling is done in polling mode. The register could be read, if the cpu_ready bit in the register ENABLE is set.

4.6 Frame interpretation

Every frame consists of a preheader, a frame header, frame data and a frame footer. A frame could be a result or a histogram frame.

Table 14: Frame

Pre header	Frame header	Frame data (Result / Histogram)	Frame footer

The size of a frame could be calculated with the equation:

Equation 1:

$$data\_frame\_size = preheader\_size + header\_size + data\_size + footer\_size$$

With,

preheader_size = 5

header_size = 16

data_size … result Frame: see equation “data size of result frame”; histogram frame = 25344

footer_size = 12

4.6.1 Preheader

The preheader contains the information of the register FIFOSTATUS and the SYSTICK registers.

The systick can be calculated with:

systick = systick_0 + (systick_1 * 256) + (systick_2 * 256 *256) + (systick_3 * 256 * 256 * 256)

4.6.2 Frame header

Every data frame has a frame header of 16 bytes. The header has the following format:

Table 15: Frame header

Byte	Bit	Content
0	7:4	Frame ID … 0x10 for result frame, 0x20 for histogram frame
0	3:0	Focal plane mode
1	-	For a result frame the result format(1), for a histogram frames the sub-frame number
2:3	-	Payload (little endian format), excluding these two bytes and previous two bytes
4:7	-	Frame number (little endian format)
8:10	-	Temperature [°Celsius] from the three temperature sensors
11	-	BDV value (Break Down Voltage), internal register value, shall be ignored by user
12:13	-	Optical reference peak from 1st measurement of this frame in 0.25mm steps (little endian format), shall be ignored by user
14:15	-	Optical reference peak from last measurement of this frame in 0.25mm steps (little endian format), shall be ignored by user

(1) For the result format description see configuration register TMF8829_CFG_RESULT_FORMAT in the datasheet.

4.6.3 Frame footer

Every data frame has a frame footer consisting of 12 bytes. The footer has the following format:

Table 16: Frame footer

Byte	Bit	Content
0:3	-	t0 integration is the TMF8829's internal timestamp when t0 integration started
4:7	-	t1 integration is the TMF8829's internal timestamp when t-last(1) integration started
8	0	Set = frame is valid
8	3	Set = Warning: SPAD CP maximum power reached; likely cause by too high ambient light; use a different operating mode, reduce number of SPADs or reduce ambient light
8	4	Set = Info: Maximum VCSEL power reached
8	5	Set = VCSEL burst limit exceeded; too high number of iterations set
8	6:7	Frame was aborted
9	2:0	Reserved
10:11	-	0xE0F7 … end of frame marker (little endian)

(1) Note that t-last means for:
8x8 or 16x16 focal plane mode: The t1 integration
32x32 focal plane mode: The t7 integration
48x32 focal plane mode: The t11 integration

4.6.4 Result frames

In focal plane mode 8x8 or 16x16 one result frame has the data of one measurement result. In focal plane mode 32x32 or 48x32 two result frames have the data of one measurement result. The result frame identifier is 0x10. The sub-frame bit shall be used to identify to which half the result belongs. The published result frame data structure depends on the setting of the focal plane mode and the frame format.

The result frame data size calculation is done with the formulas:

Equation 2: Data size of result frame

$$data\_size = number\_pixel\_frame * pixel\_size$$

Equation 3: Pixel size

$$pixel\_size = (nr\_peaks * (3 + 2 * signal\_strength)) + 2 * noise\_strength + 2 * xtalk$$

Equation 4: Number of pixels per frame

$$number\_pixel\_frame = number\_pixel \quad \text{for focal plane modes 8x8 and 16x16}$$

$$number\_pixel\_frame = number\_pixel/2 \quad \text{for focal plane modes 48x16 and 32x16}$$

4.6.4.1 Pixel alignment

The result frame has the pixel information arranged one after the other. The arrangement of pixels is done row wise.

- **8x8 mode**
  The first 8 frame pixel belongs to the first row, the second 8 pixel belongs to the second row, and so on.

Table 17: Pixel alignment in frame for 8x8 focal plane mode

Pixel 0 (x=0, y=0)	**Pixel 1 (x=1, y=0)**	…	**Pixel 7 (x=7, y=0)**	**Pixel 8 (x=0, y=1)**	…	**Pixel 15 (x=7, y=1)**	…	**Pixel 56 (x=0, y=7)**	…	**Pixel 63 (x=0, y=7)**

Figure 1: Pixel alignment in frame for 8x8 focal plane mode

8 x 8 Mode

Y \ X	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23
3	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47
6	48	49	50	51	52	53	54	55
7	56	57	58	59	60	61	62	63

- **16x16 mode**
  The first 16 frame pixel belongs to the first row, the second 16 pixel belongs to the second row, and so on.

Table 18: Pixel alignment in frame for 16x16 focal plane mode

Pixel 0	**Pixel 1**	**…**	**Pixel 15**	**Pixel 16**	**…**	**Pixel 31**	**…**	**Pixel 240**	**…**	**Pixel 255**

Figure 2: Pixel alignment in frame for 16x16 focal plane mode

16 x 16 Mode

Y \ X	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
2	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
3	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
4	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
5	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
6	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
7	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
8	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143
9	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
10	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
11	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
12	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207
13	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223
14	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239
15	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255

- **32x32 mode**
  The first frame has pixels which belong to rows with even numbers. The second frame has pixels which belong to rows with odd numbers.

Table 19: Pixel alignment in frame one for 32x32 focal plane mode

Pixel 0	**Pixel 1**	**…**	**Pixel 31**	**Pixel 64**	**…**	**Pixel 95**	**…**	**Pixel 960**	**…**	**Pixel 991**

Table 20: Pixel alignment in frame two for 32x32 focal plane mode

Pixel 32	**Pixel 33**	**…**	**Pixel 63**	**Pixel 96**	**…**	**Pixel 127**	**…**	**Pixel 992**	**…**	**Pixel 1023**

Figure 3: Pixel alignment in frame one and two for 32x32 focal plane mode

32 x 32 Mode

X / Y	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
1	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
2	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
3	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
4	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
5	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
6	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223
7	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255
8	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287
9	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319
10	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351
11	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383
12	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415
13	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447
14	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479
15	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511
16	512	513	514	515	516	517	518	519	520	521	522	523	524	525	526	527	528	529	530	531	532	533	534	535	536	537	538	539	540	541	542	543
17	544	545	546	547	548	549	550	551	552	553	554	555	556	557	558	559	560	561	562	563	564	565	566	567	568	569	570	571	572	573	574	575
18	576	577	578	579	580	581	582	583	584	585	586	587	588	589	590	591	592	593	594	595	596	597	598	599	600	601	602	603	604	605	606	607
19	608	609	610	611	612	613	614	615	616	617	618	619	620	621	622	623	624	625	626	627	628	629	630	631	632	633	634	635	636	637	638	639
20	640	641	642	643	644	645	646	647	648	649	650	651	652	653	654	655	656	657	658	659	660	661	662	663	664	665	666	667	668	669	670	671
21	672	673	674	675	676	677	678	679	680	681	682	683	684	685	686	687	688	689	690	691	692	693	694	695	696	697	698	699	700	701	702	703
22	704	705	706	707	708	709	710	711	712	713	714	715	716	717	718	719	720	721	722	723	724	725	726	727	728	729	730	731	732	733	734	735
23	736	737	738	739	740	741	742	743	744	745	746	747	748	749	750	751	752	753	754	755	756	757	758	759	760	761	762	763	764	765	766	767
24	768	769	770	771	772	773	774	775	776	777	778	779	780	781	782	783	784	785	786	787	788	789	790	791	792	793	794	795	796	797	798	799
25	800	801	802	803	804	805	806	807	808	809	810	811	812	813	814	815	816	817	818	819	820	821	822	823	824	825	826	827	828	829	830	831
26	832	833	834	835	836	837	838	839	840	841	842	843	844	845	846	847	848	849	850	851	852	853	854	855	856	857	858	859	860	861	862	863
27	864	865	866	867	868	869	870	871	872	873	874	875	876	877	878	879	880	881	882	883	884	885	886	887	888	889	890	891	892	893	894	895
28	896	897	898	899	900	901	902	903	904	905	906	907	908	909	910	911	912	913	914	915	916	917	918	919	920	921	922	923	924	925	926	927
29	928	929	930	931	932	933	934	935	936	937	938	939	940	941	942	943	944	945	946	947	948	949	950	951	952	953	954	955	956	957	958	959
30	960	961	962	963	964	965	966	967	968	969	970	971	972	973	974	975	976	977	978	979	980	981	982	983	984	985	986	987	988	989	990	991
31	992	993	994	995	996	997	998	999	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009	1010	1011	1012	1013	1014	1015	1016	1017	1018	1019	1020	1021	1022	1023

Frame 1: Frame 2:

- **48x32 mode**
  The first frame has pixels which belong to rows with even numbers. The second frame has pixels which belong to rows with odd numbers.

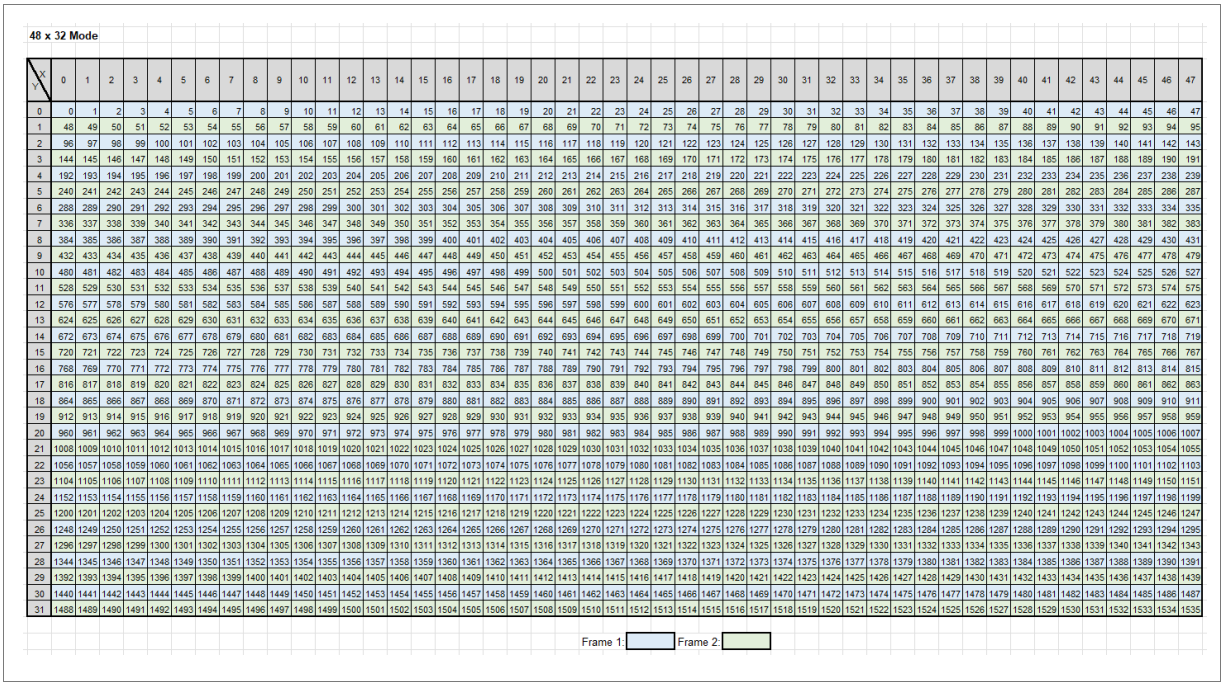
Table 21: Pixel alignment in frame one for 48x32 focal plane mode

Pixel 0	**Pixel 1**	**…**	**Pixel 48**	**Pixel 98**	**…**	**Pixel 146**	**…**	**Pixel 1470**	**…**	**Pixel 1518**

Table 22: Pixel alignment in frame two 48x32 focal plane mode

Pixel 49	**Pixel 50**	**…**	**Pixel 97**	**Pixel 147**	**…**	**Pixel 195**	**…**	**Pixel 1519**	**…**	**Pixel 1567**

Figure 4: Pixel alignment in frame one and two for 48x32 focal plane mode



4.6.4.2 Pixel data interpretation

The arrangement of the pixel data for nr_peaks = 4, signal_strength = 1, noise_strength = 1 and xtalk = 1.

Table 23: Pixel data alignment with data size

Parameter	Length [Bytes]
Noise	2
Xtalk	2
Peak 1: Distance	2
Peak 1: Snr	1
Peak 1: Signal strength	2
…	
Peak 4: Distance	2
Peak 4: Snr	1
Peak 4: Signal strength	2

Distance calculation

The decoding of the data must be done in little endian format. The distance is reported in default mode (register TMF8829_CFG_ALG_DISTANCE with value 0x01) in 0.25mm steps.

Signal strength, noise and crosstalk (xtalk)

These parameters are reported with 2 bytes. The decoding of the data must be done in little endian format.

- If bit 15 is not set, the value could be taken directly without modification (values 0 … 32767)
- If bit 15 is set, the value from bit 14 to bit 0 must be multiplied with a factor of 256)

Examples

- Data 0x7FFF: Bit 15 is not set, the value is 32767
- Data 0x8080: Bit 15 is set, the value 0x80 needs to be multiplied with 256, the result is 32768
- Data 0xFFFF: Bit 15 is set, the value 0x7FFF needs to be multiplied with 256, the result is 8,388,352

Confidence logarithmic decoding

The values 0…40 represent directly SNR. Values above 40 are exponentially scaled with a growth rate of 5.36%.

The decoding could be done with the C function:

```
#define CONF_BREAKPOINT (40)

#define EXP_GROWTH_RATE (1.053676f)


uint32_t calculateConfidence ( const uint8_t snr )

{

    uint32_t confidence = 0;

    if (snr <= CONF_BREAKPOINT)

    {

        confidence = snr;

    }

    Else /* exponential de-mapping */

    {

        const uint32_t steps = snr - CONF_BREAKPOINT;

       confidence = CONF_BREAKPOINT*pow(EXP_GROWTH_RATE,steps); // C++ pow()
function;  pow(a,b) = a^b

    }

    return confidence;

}
```

4.6.4.3 Example 1 – measurement with default settings

This example describes the instructions and the data interpretation. The register TMF8829_CFG_RESULT_FORMAT has the default value 0x01 (nr_peaks = 1) and the register TMF8829_CFG_FP_MODE the value 0x02 (16x16 focal plane mode).

Table 24: Example 1 - measurement with default settings

Function	Address	Data	Register name	Comment
Startup				
Write	0x08	0x10	CMD_STAT	CMD_MEASURE
Wait				1ms(1)
Read	0x08	0x01	CMD_STAT	STAT_ACCEPTED
Measurement				
Read	0xE1	0x01	INT_STATUS	Poll for interrupt
Read	0xFA	<frame1>	FIFOSTATUS	Read frame with length of data_frame_size(2)
Read	0xE1	0x01	INT_STATUS	Poll for interrupt
Read	0xFA	<frame2>	FIFOSTATUS	Read frame with length of data_frame_size(2)
…				
Write	0x08	0xFF	CMD_STAT	CMD_STOP
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK

(1) Maximum wait time. CMD_STAT could be polled for a STAT_ACCEPTED or STAT_OK to optimize waiting time.
(2) data_frame_size = 5 + 16 + 16 * 16 *3 + 12 = 801

The frame will have the following content:

Table 25: Example 1 - frame FP-mode 16x16 and TMF8829_CFG_RESULT_FORMAT 0x01

Frame part	Length	Byte	Content
Pre-Header	5	0 ... 4	FIFOSTATUS; SYSTICK_0 … SYSTICK_3
Frame Header	16	5 ... 20	Frame ID = 0x15; layout = 0x01; payload = 792; …
Frame Data	768	21 ... 788	Pixel 0, pixel 1 … pixel 255
Frame Footer	12	789 … 800	Frame is valid = 1; end of frame marker = 0xE0F7; …

$$\text{pixel\_size} = (\text{nr\_peaks} * (3 + 2 * \text{signal\_strength})) + 2 * \text{noise\_strength} + 2 * \text{xtalk} = 3$$

$$\text{data\_size} = \text{number\_pixel\_frame} * \text{pixel\_size}$$

The distance (2 Byte) and SNR (1 Byte) of the pixel could be calculated.

Table 26: Example 1 – pixel data

Pixel	x	y	Distance	SNR
0	0	0	Byte21 + Byte22 * 256	Byte23
1	1	0	Byte24 + Byte25 * 256	Byte26
n	x_n	y_n	Byte(n*3+21) + Byte(n*3+22) * 256	Byte(n*3+23)
255	15	15	Byte786 + Byte787 *256	Byte788

4.6.4.4 Example 2 – result format 0x39 in 8x8 focal plane mode

Pre-configuration with command CMD_LOAD_CFG_8X8 and result format 0x39 (nr_peaks=1, signal_strength=1, noise_strength=1, xtalk=1). The frame will have the following content:

Table 27: Example 2 - frame FP-mode 8x8 and TMF8829_CFG_RESULT_FORMAT 0x39

Frame part	Length	Byte	Content
Pre-Header	5	0 … 4	FIFOSTATUS; SYSTICK_0 … SYSTICK_3
Frame Header	16	5 … 20	Frame ID = 0x10; layout = 0x39; payload = 600; …
Frame Data	576	21 … 596	Pixel 0, pixel 1 … pixel 63
Frame Footer	12	597 … 608	Frame is valid = 1; end of frame marker = 0xE0F7; …

The noise(2 Byte), Xtalk(2 Byte), distance (2 Byte), SNR (1 Byte) and Signal(2Byte) of the pixel could be calculated.

Table 28: Example 2 – pixel data

Pixel	x	y	Noise	Xtalk	Distance	SNR	Signal strength
0	0	0	Byte21 + Byte22 * 256	Byte23 + Byte24 * 256	Byte25 + Byte26 * 256	Byte27	Byte28 + Byte29 * 256
n	x_n	y_n	Byte(n*9+21) + Byte(n*9+22) * 256	Byte(n*9+23) + Byte(n*9+24) * 256	Byte(n*9+25) + Byte(n*9+26) * 256	Byte(n*9+27)	Byte(n*9+28) + Byte(n*9+29) * 256
63	8	8	Byte588 + Byte589 * 256	Byte590 + Byte591 * 256	Byte592 + Byte593 * 256	Byte594	Byte595 + Byte596 * 256

4.6.4.5 Example 3 – result format 0x11 in 48x32 focal plane mode

Pre-configuration with command CMD_LOAD_CFG_48X32 and result format 0x11 (nr_peaks=1, signal_strength=0, noise_strength=1, xtalk=0)

Table 29: Example 3 - measurement with result format 0x11 and 48x32 focal plane mode

Function	Address	Data	Register name	Comment
Startup				
Wait				3ms
Write	0x08	0xFF	CMD_STAT	CMD_LOAD_CFG_48X32
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK
Write	0x2A	0x11	TMF8829_CFG_RESULT_FORMAT	Change result format
Write	0x08	0x10	CMD_STAT	CMD_WRITE_PAGE_AND_ MEASURE
Wait				1ms(1)
Read	0x08	0x01	CMD_STAT	STAT_ACCEPTED
Measurement				
Read	0xE1	0x01	INT_STATUS	Poll for interrupt
Read	0xFA	<frame1>	FIFOSTATUS	Read frame1(2), length = 3873(3)
Read	0xE1	0x01	INT_STATUS	Poll for interrupt
Read	0xFA	<frame2>	FIFOSTATUS	Read frame2(2), length = 3873(3)
…				
Write	0x08	0xFF	CMD_STAT	CMD_STOP
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK

(1) Maximum wait time. CMD_STAT could be polled for STAT_OK or STAT_ACCEPTED to optimize waiting time.
(2) Two frames belong to one measurement result.
(3) data_frame_size = 5 + 16 + 24 * 32 *5 + 12 = 3873

The frames will have the following content:

Table 30: Example 3 - first frame FP-mode 48x32 and TMF8829_CFG_RESULT_FORMAT 0x11

Frame part	Length	Byte	Content
Pre-header	5	0 ... 4	FIFOSTATUS; SYSTICK_0 … SYSTICK_3
Frame header	16	5 ... 20	Frame ID = 0x15; layout = 0x11; payload = 3864; …
Frame data	3840	21 ... 3860	Pixel 0, pixel 1 … pixel 767
Frame footer	12	3861 … 3872	Frame is valid = 1; end of frame marker = 0xE0F; …

Table 31: Example 3 - second frame FP-mode 48x32 and TMF8829_CFG_RESULT_FORMAT 0x11

Frame part	Length	Byte	Content
Pre-header	5	0 ... 4	FIFOSTATUS; SYSTICK_0 … SYSTICK_3
Frame header	16	5 ... 20	Frame ID = 0x15; Layout = 0x81; Payload = 3864; …
Frame data	3840	21 ... 3860	Pixel 768, Pixel 769 … Pixel 1535
Frame footer	12	3861 … 3872	Frame is valid = 1; end of frame marker = 0xE0F7; …

Table 32: Example 3 – pixel data frame 1, only pixels with even y-coordinate

Pixel	x	y	Frame pixel	Noise	Distance	SNR
0	0	0	0	Byte21 + Byte22 * 256	Byte23 + Byte24 * 256	Byte25
n	x_n	y_n	$m^{(1)}$	Byte(m*5+21) + Byte(m*5+22) * 256	Byte(m*5+23) + Byte(m*5+24) * 256	Byte(m*5+25)
1487	47	30	767	Byte3856 + Byte3857 * 256	Byte3858 + Byte3859 * 256	Byte3860

(1) Calculation of m = round_down(n / 96) * 48 + mod(n / 48)

Table 33: Example 3 – pixel data frame 2, only pixels with odd y-coordinate

Pixel	x	y	Frame pixel	Noise	Distance	SNR
48	0	1	0	Byte21 + Byte22 * 256	Byte23 + Byte24 * 256	Byte25
n	x_n	y_n	$m^{(1)}$	Byte(m*5+21) + Byte(m*5+22) * 256	Byte(m*5+23) + Byte(m*5+24) * 256	Byte(m*5+25)
1535	47	31	767	Byte3856 + Byte3857 * 256	Byte3858 + Byte3859 * 256	Byte3860

(1) Calculation of m = (round_down(n / 96) +1) * 48 + mod(n / 48)

4.6.5 Histogram frames

The histogram frame identifier is 0x20. The histogram data in the histogram frame (25377 byte with preheader) has always the same size with 25344 bytes. Therefore, the payload is 25368, which includes 12-header and 12-footer bytes. The second byte in the histogram frame header contains the sub-frame number. The measurement with histogram frames and the mapping of the histograms depends on the operating mode. The description is done separately for every focal plane mode.

Information:

- RP … Reference Pixel
- P(x=x0|...|xn, y = y0|…|yn) … represents all pixel with a coordinate from x and y

4.6.5.1 8x8 focal plane mode

A measurement with histogram dumping contains two histogram frames followed by a result frame. The first four histograms in a frame are reference pixel histograms with a bin size of 64, followed by pixel histograms with a bin size of 256. The first frame contains the histograms of the lift site pixels. The second frame contains the histograms of the right site pixels. Every bin is reported in a size of 3 bytes in little endian format. The first 3 bytes are bin 0, the next 3 bytes are bin 1 and so on.

Table 34: Measurement frames in 8x8 mode with histograms

Num	Frame	Content
1	Histogram T0	RP 0..3; P(x = 0|1|2|3; y = 0|1|2|3|4|5|6|7)
2	Histogram T1	RP 0..3; P(x = 4|5|6|7; y = 0|1|2|3|4|5|6|7)
3	Result 1	See result frame coding

Table 35: Histogram T0 frame data alignment in 8x8 mode

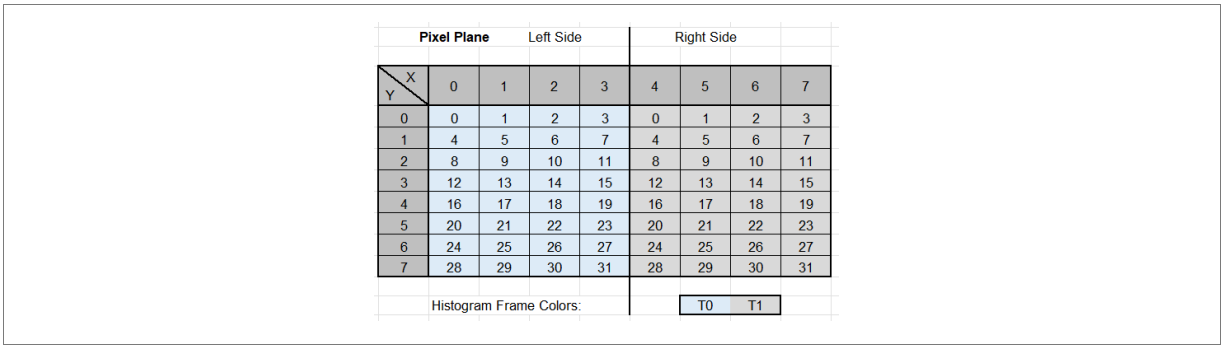
Frame pixel	Pixel	x	y	Bin size	Histogram size [Bytes]	Byte number in frame data
RP1	RP1			64	192 = 64 * 3	0 : 191
..	..					
RP4	RP4			64	192	576 : 767
0	0	0	0	256	768 = 256 *3	768 : 1535
…	…					
n	$y_n * 8 + x_n$	x_n	y_n	256	768	(768*(n+1)) : (768*(n+2)-1)
31	59	3	7	256	768	24576 : 25343

Table 36: Histogram T1 frame data alignment in 8x8 mode

Frame pixel	Pixel	x	y	Bin size	Histogram size [Bytes]	Byte number in frame data
RP1	RP1			64	192 = 64 * 3	0 : 191
..	..					
RP4	RP4			64	192	576 : 767
0	4	4	0	256	768 = 256 *3	768 : 1535
…	…					
n	$y_n * 8 + x_n$	x_n	y_n	256	768	(768*(n+1)) : (768*(n+2)-1)
31	63	7	7	256	768	24576 : 25343

The following figure shows to which coordinate on the focal plane a frame pixel histogram belongs:

Figure 5: Histogram arrangement in 8x8 mode



4.6.5.2 16x16 focal plane mode

A measurement with histogram dumping contains two histogram frames followed by a result frame. The first four histograms in a frame are reference pixel histograms with a bin size of 64, followed by pixel histograms with a bin size of 64. The first frame contains the histograms of the lift site pixels. The second frame contains the histograms of the right site pixels. Every bin is reported in a size of 3 bytes in little endian format. The first 3 bytes are bin 0, the next 3 bytes are bin 1 and so on.

Table 37: Measurement frames in 16x16 mode with histograms

Num	Frame	Content
1	Histogram T0	RP 0..3; P(x = 0|1|2|3|4|5|6|7; y = 0|1|2|3|4|5|6|7|8|9|10|11|12|13|14|15)
2	Histogram T1	RP 0..3; P(x = 8|9|10|11|12|13|14|15; y = 0|1|2|3|4|5|6|7|8|9|10|11|12|13|14|15)
3	Result 1	See result frame coding

Table 38: Histogram T0 frame data alignment in 16x16 mode

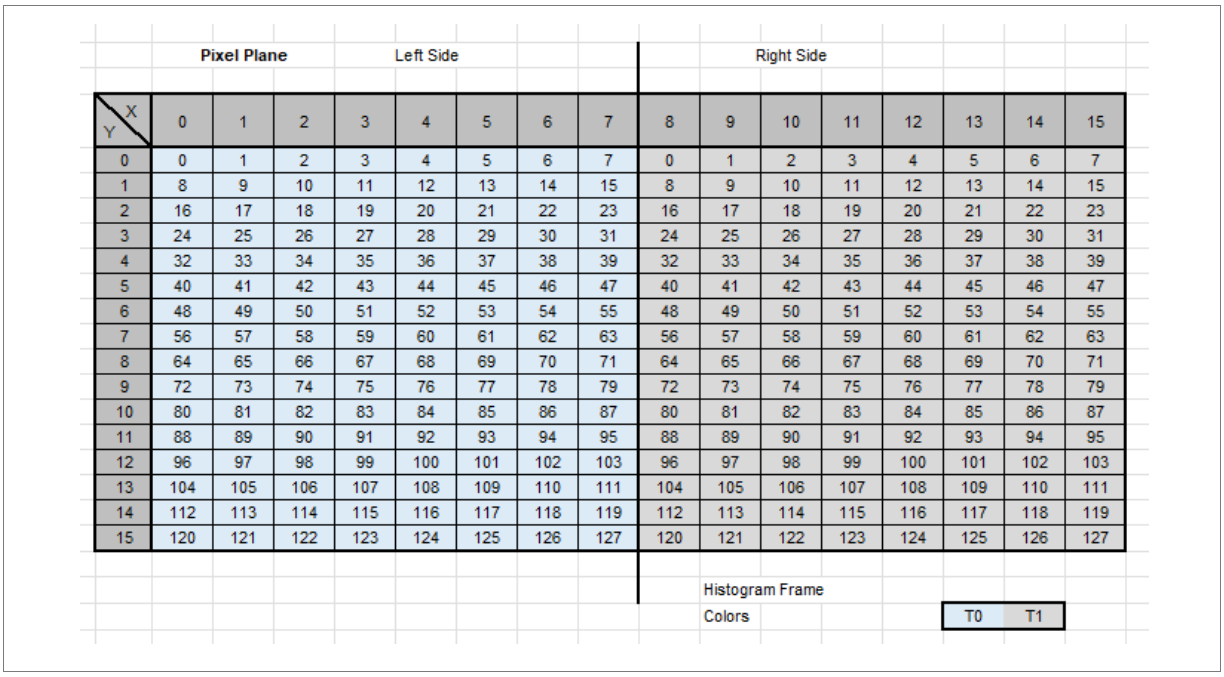
Frame pixel	Pixel	x	y	Bin size	Histogram size [Bytes]	Byte number in frame data
RP1	RP1			64	192 = 64 * 3	0 : 191
..	..					
RP4	RP4			64	192	576 : 767
0	0	0	0	64	192	768 : 959
…	…					
n	$y_n * 8 + x_n$	x_n	y_n	64	192	(192*(n+4)) : (192*(n+5)-1)
127	247	7	15	64	192	25152 : 25343

Table 39: Histogram T1 frame data alignment in 16x16 mode

Frame pixel	Pixel	x	y	Bin size	Histogram size [Bytes]	Byte number in frame data
RP1	RP1			64	192 = 64 * 3	0 : 191
..	..					
RP4	RP4			64	192	576 : 767
0	0	0	8	64	192	768 : 959
…	…					
n	$y_n * 8 + x_n$	x_n	y_n	64	192	(192*(n+4)) : (192*(n+5)-1)
127	247	15	15	64	192	25152 : 25343

The following figure shows to which coordinate on the focal plane a frame pixel histogram belongs.

Figure 6: Histogram arrangement in 16x16 mode



	Pixel Plane				Left Side							Right Side				
X / Y	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23	16	17	18	19	20	21	22	23
3	24	25	26	27	28	29	30	31	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39	32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47	40	41	42	43	44	45	46	47
6	48	49	50	51	52	53	54	55	48	49	50	51	52	53	54	55
7	56	57	58	59	60	61	62	63	56	57	58	59	60	61	62	63
8	64	65	66	67	68	69	70	71	64	65	66	67	68	69	70	71
9	72	73	74	75	76	77	78	79	72	73	74	75	76	77	78	79
10	80	81	82	83	84	85	86	87	80	81	82	83	84	85	86	87
11	88	89	90	91	92	93	94	95	88	89	90	91	92	93	94	95
12	96	97	98	99	100	101	102	103	96	97	98	99	100	101	102	103
13	104	105	106	107	108	109	110	111	104	105	106	107	108	109	110	111
14	112	113	114	115	116	117	118	119	112	113	114	115	116	117	118	119
15	120	121	122	123	124	125	126	127	120	121	122	123	124	125	126	127

Histogram Frame Colors	T0	T1

4.6.5.3 32x32 focal plane mode

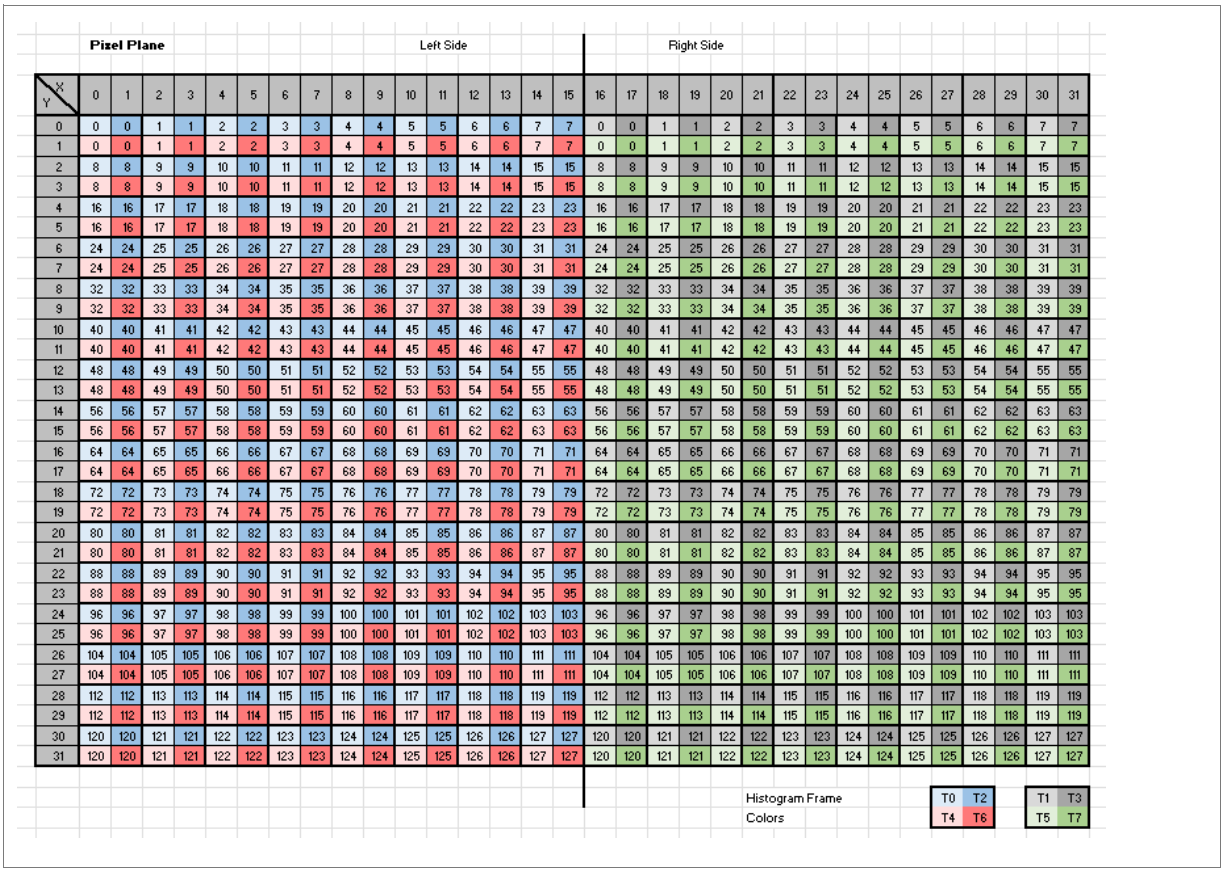
A measurement with histogram dumping contains eight histogram frames and two result frames. The first four histograms in a frame are reference pixel histograms with a bin size of 64, followed by pixel histograms with a bin size of 64. Every bin is reported in a size of 3 bytes in little endian format. The first 3 bytes are bin 0, the next 3 bytes are bin 1 and so on.

Table 40: Measurement frames in 32x32 mode with histograms

Num	Frame	Content
1	Histogram T0	RP 0..3; P(x = 0|3|6|9|12|15|18|21; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
2	Histogram T1	RP 0..3; P(x = 24|27|30|33|36|39|42|45; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
3	Histogram T2	RP 0..3; P(x = 1|4|7|10|13|16|19|22; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
4	Histogram T3	RP 0..3; P(x = 25|28|31|34|37|40|43|46; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
5	Result 1	See result frame coding
6	Histogram T4	RP 0..3; P(x = 0|3|6|9|12|15|18|21; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
7	Histogram T5	RP 0..3; P(x = 24|27|30|33|36|39|42|45; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
8	Histogram T6	RP 0..3; P(x = 1|4|7|10|13|16|19|22; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
9	Histogram T7	RP 0..3; P(x = 25|28|31|34|37|40|43|46; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
10	Result 2	See result frame coding

The following figure shows to which coordinate on the focal plane a frame pixel histogram belongs. See also chapter “C-function for mapping of histograms”.

Figure 7: Histogram arrangement in 32x32 mode



Pixel Plane																	Left Side															Right Side															
Y \ X	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	0	1	1	2	2	3	3	4	4	5	5	6	6	7	7	0	0	1	1	2	2	3	3	4	4	5	5	6	6	7	7
1	0	0	1	1	2	2	3	3	4	4	5	5	6	6	7	7	0	0	1	1	2	2	3	3	4	4	5	5	6	6	7	7
2	8	8	9	9	10	10	11	11	12	12	13	13	14	14	15	15	8	8	9	9	10	10	11	11	12	12	13	13	14	14	15	15
3	8	8	9	9	10	10	11	11	12	12	13	13	14	14	15	15	8	8	9	9	10	10	11	11	12	12	13	13	14	14	15	15
4	16	16	17	17	18	18	19	19	20	20	21	21	22	22	23	23	16	16	17	17	18	18	19	19	20	20	21	21	22	22	23	23
5	16	16	17	17	18	18	19	19	20	20	21	21	22	22	23	23	16	16	17	17	18	18	19	19	20	20	21	21	22	22	23	23
6	24	24	25	25	26	26	27	27	28	28	29	29	30	30	31	31	24	24	25	25	26	26	27	27	28	28	29	29	30	30	31	31
7	24	24	25	25	26	26	27	27	28	28	29	29	30	30	31	31	24	24	25	25	26	26	27	27	28	28	29	29	30	30	31	31
8	32	32	33	33	34	34	35	35	36	36	37	37	38	38	39	39	32	32	33	33	34	34	35	35	36	36	37	37	38	38	39	39
9	32	32	33	33	34	34	35	35	36	36	37	37	38	38	39	39	32	32	33	33	34	34	35	35	36	36	37	37	38	38	39	39
10	40	40	41	41	42	42	43	43	44	44	45	45	46	46	47	47	40	40	41	41	42	42	43	43	44	44	45	45	46	46	47	47
11	40	40	41	41	42	42	43	43	44	44	45	45	46	46	47	47	40	40	41	41	42	42	43	43	44	44	45	45	46	46	47	47
12	48	48	49	49	50	50	51	51	52	52	53	53	54	54	55	55	48	48	49	49	50	50	51	51	52	52	53	53	54	54	55	55
13	48	48	49	49	50	50	51	51	52	52	53	53	54	54	55	55	48	48	49	49	50	50	51	51	52	52	53	53	54	54	55	55
14	56	56	57	57	58	58	59	59	60	60	61	61	62	62	63	63	56	56	57	57	58	58	59	59	60	60	61	61	62	62	63	63
15	56	56	57	57	58	58	59	59	60	60	61	61	62	62	63	63	56	56	57	57	58	58	59	59	60	60	61	61	62	62	63	63
16	64	64	65	65	66	66	67	67	68	68	69	69	70	70	71	71	64	64	65	65	66	66	67	67	68	68	69	69	70	70	71	71
17	64	64	65	65	66	66	67	67	68	68	69	69	70	70	71	71	64	64	65	65	66	66	67	67	68	68	69	69	70	70	71	71
18	72	72	73	73	74	74	75	75	76	76	77	77	78	78	79	79	72	72	73	73	74	74	75	75	76	76	77	77	78	78	79	79
19	72	72	73	73	74	74	75	75	76	76	77	77	78	78	79	79	72	72	73	73	74	74	75	75	76	76	77	77	78	78	79	79
20	80	80	81	81	82	82	83	83	84	84	85	85	86	86	87	87	80	80	81	81	82	82	83	83	84	84	85	85	86	86	87	87
21	80	80	81	81	82	82	83	83	84	84	85	85	86	86	87	87	80	80	81	81	82	82	83	83	84	84	85	85	86	86	87	87
22	88	88	89	89	90	90	91	91	92	92	93	93	94	94	95	95	88	88	89	89	90	90	91	91	92	92	93	93	94	94	95	95
23	88	88	89	89	90	90	91	91	92	92	93	93	94	94	95	95	88	88	89	89	90	90	91	91	92	92	93	93	94	94	95	95
24	96	96	97	97	98	98	99	99	100	100	101	101	102	102	103	103	96	96	97	97	98	98	99	99	100	100	101	101	102	102	103	103
25	96	96	97	97	98	98	99	99	100	100	101	101	102	102	103	103	96	96	97	97	98	98	99	99	100	100	101	101	102	102	103	103
26	104	104	105	105	106	106	107	107	108	108	109	109	110	110	111	111	104	104	105	105	106	106	107	107	108	108	109	109	110	110	111	111
27	104	104	105	105	106	106	107	107	108	108	109	109	110	110	111	111	104	104	105	105	106	106	107	107	108	108	109	109	110	110	111	111
28	112	112	113	113	114	114	115	115	116	116	117	117	118	118	119	119	112	112	113	113	114	114	115	115	116	116	117	117	118	118	119	119
29	112	112	113	113	114	114	115	115	116	116	117	117	118	118	119	119	112	112	113	113	114	114	115	115	116	116	117	117	118	118	119	119
30	120	120	121	121	122	122	123	123	124	124	125	125	126	126	127	127	120	120	121	121	122	122	123	123	124	124	125	125	126	126	127	127
31	120	120	121	121	122	122	123	123	124	124	125	125	126	126	127	127	120	120	121	121	122	122	123	123	124	124	125	125	126	126	127	127

Histogram Frame Colors

T0	T2
T4	T6

T1	T3
T5	T7

4.6.5.4 48x32 focal plane mode

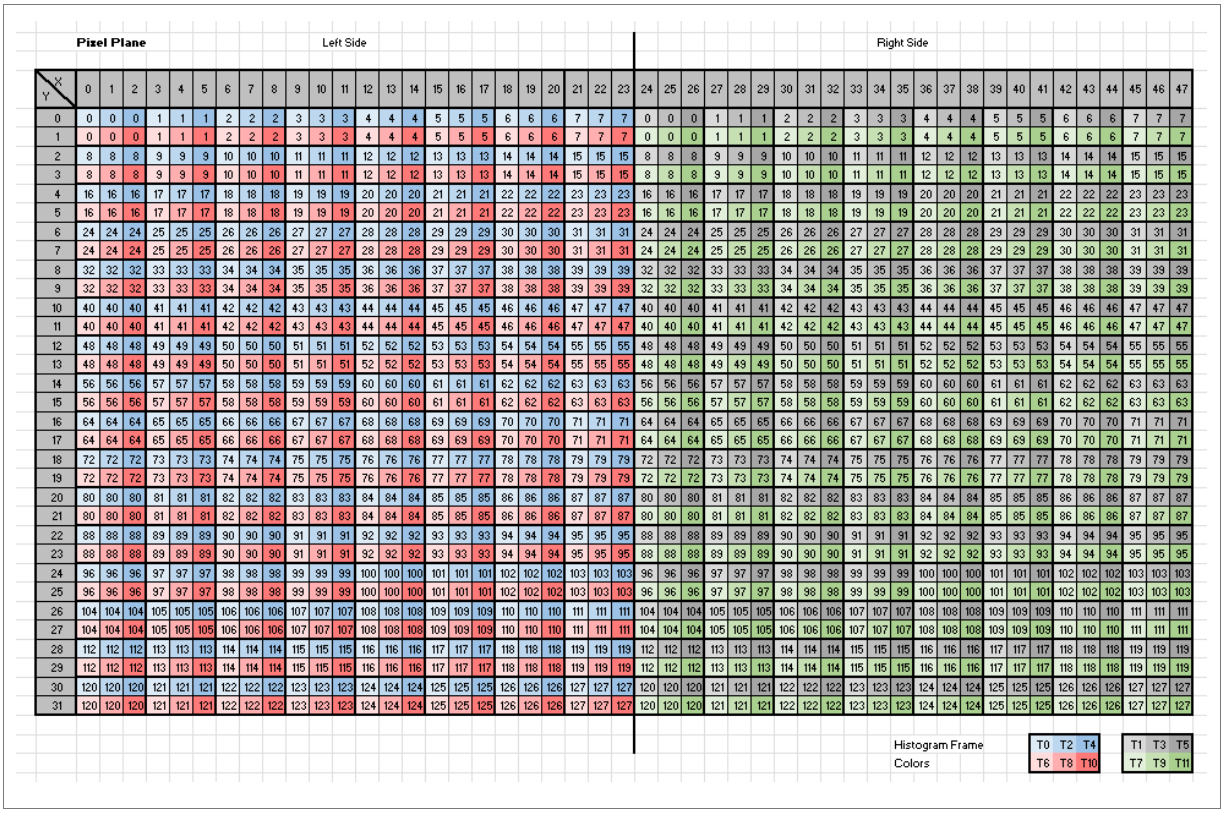
A measurement with histogram dumping contains twelve histogram frames and two result frames. The first four histograms in a frame are reference pixel histograms with a bin size of 64, followed by pixel histograms with a bin size of 64. Every bin is reported in a size of 3 bytes in little endian format. The first 3 bytes are bin 0, the next 3 bytes are bin 1 and so on.

Table 41: Measurement frames in 48x32 mode with histograms

Num	Frame	Content
1	Histogram T0	RP 0..3; P(x = 0|3|6|9|12|15|18|21; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
2	Histogram T1	RP 0..3; P(x = 24|27|30|33|36|39|42|45; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
3	Histogram T2	RP 0..3; P(x = 1|4|7|10|13|16|19|22; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
4	Histogram T3	RP 0..3; P(x = 25|28|31|34|37|40|43|46; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
5	Histogram T4	RP 0..3; P(x = 2|5|8|11|14|17|20|23; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
6	Histogram T5	RP 0..3; P(x = 26|29|32|35|38|41|44|47; y = 0|2|4|6|8|10|12|14|16|18|20|22|24|26|28|30)
7	Result 1	See result frame coding
8	Histogram T6	RP 0..3; P(x = 0|3|6|9|12|15|18|21; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
9	Histogram T7	RP 0..3; P(x = 24|27|30|33|36|39|42|45; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
10	Histogram T8	RP 0..3; P(x = 1|4|7|10|13|16|19|22; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
11	Histogram T9	RP 0..3; P(x = 25|28|31|34|37|40|43|46; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
12	Histogram T10	RP 0..3; P(x = 2|5|8|11|14|17|20|23; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
13	Histogram T11	RP 0..3; P(x = 26|29|32|35|38|41|44|47; y = 1|3|5|7|9|11|13|15|17|19|21|23|25|27|29|31)
14	Result 2	See result frame coding

The following figure shows to which coordinate on the focal plane a frame pixel histogram belongs. See also C-function for mapping of histograms.

Figure 8: Histogram arrangement in 48x32 mode



4.6.5.5 Example 1 - measurement with histograms with focal plane mode 16x16

This example describes a measurement with FP-mode 16x16 and default settings but with histogram dumping. For mapping of histogram frames see chapter 16x16 focal plane mode.

Table 42: Example 1 - measurement with default settings

Function	Address	Data	Register name	Comment
Startup				
Write	0x08	0x16	CMD_STAT	CMD_LOAD_CONFIG_ PAGE
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK
Write	0x2B	0x01	TMF8829_CFG_DUMP_HISTOGRAMS	Provide raw histograms
Write	0x08	0x14	CMD_STAT	CMD_WRITE_PAGE_AND_ MEASURE
Wait				1ms(1)
Read	0x08	0x01	CMD_STAT	STAT_ACCEPTED
Measurement				
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3 - histogram INT)
Read	0xFA	<frame1 >	FIFOSTATUS	Read histogram frame T0 (length = 25377)
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3 - histogram INT)
Read	0xFA	<frame2>	FIFOSTATUS	Read histogram frame T1 (length = 25377)
Read	0xE1	0x01	INT_STATUS	Poll for interrupt (int1 - result INT)
Read	0xFA	<frame3>	FIFOSTATUS	Read frame (length=801 in default mode)
…				
Write	0x08	0xFF	CMD_STAT	CMD_STOP
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK

(1) Maximum wait time. CMD_STAT could be polled for STAT_OK or STAT_ACCEPTED to optimize waiting time.

4.6.5.6 Example 2 - measurement with histograms in operating mode 32x32

This example describes a measurement in FP-mode 32x32 with preconfigure command CMD_LOAD_CFG_32X32 and histogram dumping. For mapping of histogram frames see chapter 32x32 focal plane mode.

Table 43: Example 2 - measurement with histograms in operating mode 32x32

Function	Address	Data	Register name	Comment
Startup				
Write	0x08	0x45	CMD_STAT	CMD_LOAD_CFG_32X32
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK
Write	0x2B	0x01	TMF8829_CFG_DUMP_HISTOGRAMS	Provide raw histograms
Write	0x08	0x14	CMD_STAT	CMD_WRITE_PAGE_AND_ MEASURE
Wait				1ms(1)
Read	0x08	0x01	CMD_STAT	STAT_ACCEPTED
Measurement				
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3)
Read	0xFA	<frame1>	FIFOSTATUS	Read histogram frame T0 (length=25377)
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3)
Read	0xFA	<frame2>	FIFOSTATUS	Read histogram frame T1 (length=25377)
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3)
Read	0xFA	<frame3>	FIFOSTATUS	Read histogram frame T2 (length=25377)
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3)
Read	0xFA	<frame4>	FIFOSTATUS	Read histogram frame T3 (length=25377)
Read	0xE1	0x01	INT_STATUS	Poll for interrupt (int1)
Read	0xFA	<frame5>	FIFOSTATUS	Read frame (length=data_frame_size(2))
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3)
Read	0xFA	<frame1>	FIFOSTATUS	Read histogram frame T4 (length=25377)
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3)
Read	0xFA	<frame2>	FIFOSTATUS	Read histogram frame T5 (length=25377)
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3)

Function	Address	Data	Register name	Comment
Read	0xFA	<frame3>	FIFOSTATUS	Read histogram frame T6 (length=25377)
Read	0xE1	0x08	INT_STATUS	Poll for interrupt (int3)
Read	0xFA	<frame4>	FIFOSTATUS	Read histogram frame T7 (length=25377)
Read	0xE1	0x01	INT_STATUS	Poll for interrupt (int1)
Read	0xFA	<frame5>	FIFOSTATUS	Read frame (length=data_frame_size(2))
Write	0x08	0xFF	CMD_STAT	CMD_STOP
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK

(1) Maximum wait time. CMD_STAT could be polled for a STAT_OK or STAT_ACCEPTED to optimize waiting time.
(2) For calculation see chapter “Result frames”.

4.6.5.7 C-function for mapping of histograms

The c-function to map between location and the histograms:

Input

- histogramNumber:
  - histogramNumber in a sorted array of histograms, where histogram = 0 is at x = 0,y = 0; histogram = 1 at x = 1,y = 0 and so on
  - histogramNumber = x + y * NUMBER_OF_COLS where NUMBER_OF_COLS is 8, 16, 32 or 48
- cols:
  - Operating mode where COLS_8 for 8x8, COLS_16 for 16x16, COLS_32 for 32x32 and COLS_48 for 48x32 mode

Return value

- mappedNumber: Histogram position in an array of histograms as delivered by the firmware (multiplexed in sub frames)

```
#define COLS_4  (4)
#define COLS_8  (8)
#define COLS_16 (16)
#define COLS_32 (32)
#define COLS_48 (48)
#define ROWS_8  (8)
#define ROWS_16 (16)
#define ROWS_32 (32)
uint32_t mapHistogramNumber( const uint32_t histogramNumber,
                             const uint32_t cols )
{
    const uint32_t x = histogramNumber % cols;
    const uint32_t y = histogramNumber / cols;
    const uint32_t histogramsPerPacket = COLS_8 * ROWS_16;

    uint32_t mappedNumber = histogramNumber;

    if (cols == COLS_8)
    {
        mappedNumber = (x % COLS_4) + ((x % COLS_8) >= COLS_4) *
                       (COLS_4 * ROWS_8) + (y * COLS_4);
    }
    if (cols == COLS_16)
    {
        mappedNumber = (x % COLS_8) + ((x % COLS_16) >= COLS_8) *
                       (COLS_8 * ROWS_16) + (y * COLS_8);
    }
    if (cols == COLS_32)
    {
        mappedNumber = ((x / 2) % COLS_8) + ((y / 2) % ROWS_16) * COLS_8;
```

```
        mappedNumber += ( x >= COLS_32 / 2 ) * histogramsPerPacket;
        mappedNumber += ( x % 2 )            * histogramsPerPacket * 2;
        mappedNumber += ( y % 2 )            * histogramsPerPacket * 4;
    }
    if (cols == COLS_48)
    {
        mappedNumber = ((x / 3) % COLS_8) + ((y / 2) % ROWS_16) * COLS_8;
        mappedNumber += (x >= COLS_48 / 2) * histogramsPerPacket;
        mappedNumber += ((x % 3 ) == 1) * histogramsPerPacket * 2;
        mappedNumber += ((x % 3 ) == 2) * histogramsPerPacket * 4;
        mappedNumber += (y % 2 ) * histogramsPerPacket * 6;
    }
    return mappedNumber;
}
```

4.7 Dual mode

The dual mode combines in one measurement the high accuracy mode and the default mode. In focal plane mode 8x8, the high accuracy mode could also be combined with the long range mode. A description is available in the datasheet.

For result frames, the frame number will increase by two for every measurement. If histograms are dumped the number of histogram frames will double. The first set of the frames belongs to the high accuracy mode. The second set of the frames belongs to the default mode or the long range mode.

Table 44: Measurement frames in 16x16 dual mode with histograms

Num	Frame	Content
1	Histogram T0	High Accuracy mode: RP 0..3; P(x = 0|1|2|3|4|5|6|7; y = 0|1|2|3|4|5|6|7|8|9|10|11|12|13|14|15)
2	Histogram T1	High Accuracy mode: RP 0..3; P(x = 8|9|10|11|12|13|14|15; y = 0|1|2|3|4|5|6|7|8|9|10|11|12|13|14|15)
3	Histogram T0	Default mode: RP 0..3; P(x = 0|1|2|3|4|5|6|7; y = 0|1|2|3|4|5|6|7|8|9|10|11|12|13|14|15)
4	Histogram T1	Default mode: RP 0..3; P(x = 0|1|2|3|4|5|6|7; y = 0|1|2|3|4|5|6|7|8|9|10|11|12|13|14|15)
5	Result 1	See result frame coding, the frame number increased by two

4.8 3D point cloud calculation

The python function calculates the correction factor for the (virtual) pixel based on the number of spad-per-pixel, and the x / y coordinates of the pixel. Note that these are virtual pixels.

- E.g. in 8x8 mode a virtual pixel has the indices 0..7 and spad_per_pixel_x = 6, spad_per_pixel_y = 4
- E.g. in 16x16 mode a virtual pixel has the indices 0..15 and spad_per_pixel_x = 3, spad_per_pixel_y = 2
- E.g. in 32x32 mode a virtual pixel has the indices 0..31 and spad_per_pixel_x = 1.5, spad_per_pixel_y = 1
- E.g. in 48x32 mode a virtual pixel has the indices 0..47 and spad_per_pixel_x = 1, spad_per_pixel_y = 1

Args

- pixel_x: x-index of the virtual pixel (0..7, 0..15, 0..31, 0..47)
- pixel_y: y-index of the virtual pixel (0..7, 0..15, 0..31)
- fp_mode: 0/1 = 8x8, 2 = 16x16, 3/4 = 32x32, else 48x32

Returns

- Return a correction factor for x/y position for a single pixel

```
def zCorrectionFactorSimple(pixel_x, pixel_y, fp_mode ):
    import math
    if fp_mode == 0 or fp_mode == 1:
        X = 8
        Y = 8
    elif fp_mode == 2:
        X = 16
        Y = 16
    elif fp_mode == 3 or fp_mode == 4:
        X = 32
        Y = 32
    else:
        X = 48
        Y = 32
    spanX = X * 3.0 / 4.0
    spanY = Y
    x = ( pixel_x - ((X-1)/2) - 0.5 ) / spanX
    y = ( pixel_y - ((Y-1)/2) - 0.5 ) / spanY
    return math.sqrt( 1 + x*x + y*y )
```

4.9 Motion detection

The motion detection feature is enabled and configured via the configuration page. Motion detection is done with a post processing algorithm and needs to be enabled within the register TMF8829_CFG_POST_PROCESSING. The motion detection configuration parameters are motion_distance, detect_snr and release_snr. It is only enabled when interrupt persistence mode is active. The int-persistence parameters for the threshold minimum and maximum distance should be set to the desired values. The following sequence gives an overview for the configuration.

Table 45: Sequence for motion detection configuration with example values

Function	Address	Data	Register name	Comment
Write	0x08	0x16	CMD_STAT	CMD_LOAD_CONFIG_PAGE
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK
Write	0x68	0x00	TMF8829_CFG_INT_THRESHOLD_LOW_LSB	Set minimum distance to 128 mm
Write	0x69	0x02	TMF8829_CFG_INT_THRESHOLD_LOW_MSB	
Write	0x6A	0xA0	TMF8829_CFG_INT_THRESHOLD_HIGH_LSB	Set maximum distance to 1000 mm
Write	0x6B	0x0F	TMF8829_CFG_INT_THRESHOLD_HIGH_MSB	
Write	0x6C	0x02	TMF8829_CFG_INT_PERSISTENCE	Persistence to 2
Write	0x6D	0x01	TMF8829_CFG_POST_PROCESSING	Enable motion detection feature
Write	0xB0	0xD2	TMF8829_CFG_MOTION_DETECT_DISTANCE_LSB	Set distance delta considered as motion to 244mm
Write	0xB1	0x03	TMF8829_CFG_MOTION_DETECT_DISTANCE_MSB	
Write	0xB2	0x0C	TMF8829_CFG_MOTION_DETECT_SNR	Detect snr to 12
Write	0xB3	0x06	TMF8829_CFG_MOTION_RELEASE_SNR	Release snr to 6
Write	0x08	0x15	CMD_STAT	CMD_WRITE_PAGE
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK

(1) Maximum wait time. CMD_STAT could be polled for a STAT_ACCEPTED.

4.10 Proximity detection

The proximity detection feature is configured via the configuration page. The register TMF8829_CFG_PROX_DISTANCE has the default value 0 and will not do proximity detection. The desired value larger 0 must be set to enable this feature. A proximity detection results in an interrupt and can be read out from the register INT_STATUS bit int2. To enable also the interrupt line, the bit int2_enab in the register INT_ENAB must be set.

Attention:

For close distances, a short range mode or a default mode combined with the dual mode setting shall be used.

4.11 Oscillator drift correction

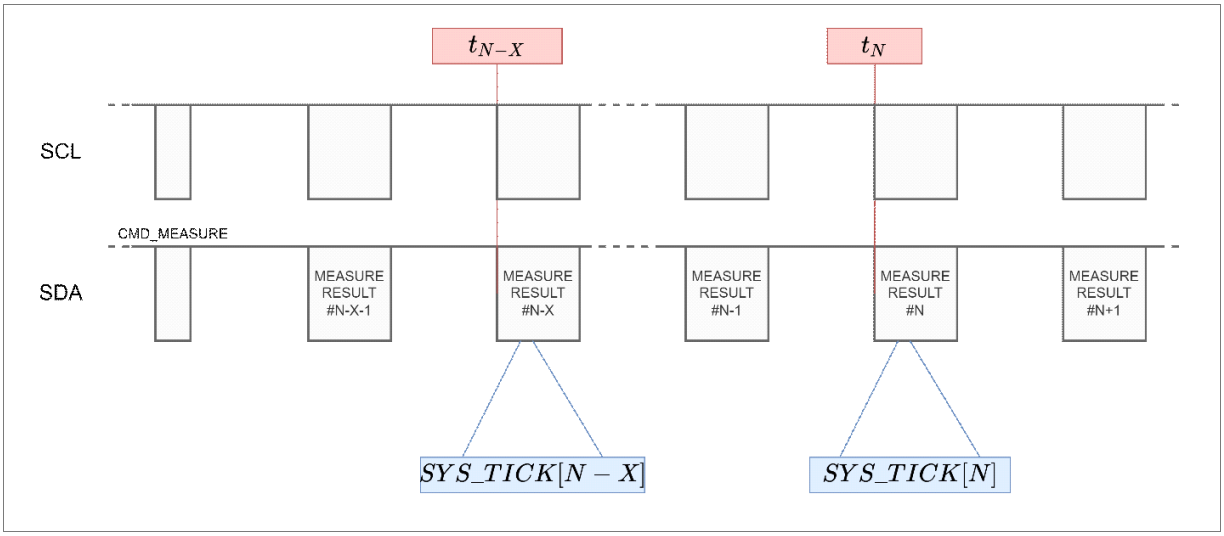
The internal oscillator of the TMF8829 can drift over time due to e.g., temperature changes. This drift has a negative impact on the measurement accuracy.

The timestamp is part of the measurement result structure and located in the preheader (SYSTICK registers). It is reported by the TMF8829 in registers SYS_CLOCK_0, SYS_CLOCK_1, SYS_CLOCK_2, SYS_CLOCK_3 in little endian format.

The host can compare the result timestamp difference against a precise reference clock (e.g., the host system tick). The host can then compensate for the measurement distance error caused by the internal oscillator drift.

The TMF8829 reports its internal timestamp since it has been activated (PON = 1). The resolution of the timestamp is 1/125 kHz = 8 microseconds. The reported value is an unsigned 32-bit value that wraps around approximately every 9.54 hours. If the LSB of the timestamp is 0, the timestamp value is invalid and should not be considered for oscillator drift correction.

Figure 9: Read the timestamp to calculate the oscillator drift correction



The host can compute the oscillator drift correction by comparing two TMF8829 timestamps with timestamps of the I²C transfers:

Equation 5: Calculation of correction factor

$$correction\_factor = \frac{t_N - t_{N-x}}{(SYS\_TICK[N] - SYSTICK[N-X]) * 8\mu s}$$

The host can then correct the reported distance by the drift correction:

Equation 6: Calculate actual distance with correction factor

$$actual\_distance[t_N] = reported\_distance[t_N] * correction\_factor[t_N]$$

4.12 Oscillator tuning

Two application commands are provided to tune the oscillator faster or slower. To tune the oscillator faster use the application command CMD_OSC_TUNE_UP which will increase the power to the oscillator. To tune the oscillator slower use the application command CMD_OSC_TUNE_DOWN which will decrease the power to the oscillator.

Information:

- The tuning is not monotonic increasing or decreasing.

Table 46: Oscillator tuning commands

Function	Address	Data	Register name	Comment
Tune oscillator up				
Write	0x08	0x1E	CMD_STAT	CMD_OSC_TUNE_UP
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK
Tune oscillator down				
Write	0x08	0x1F	CMD_STAT	CMD_OSC_TUNE_DOWN
Wait				1ms(1)
Read	0x08	0x00	CMD_STAT	STAT_OK

(1) Maximum wait time. CMD_STAT could be polled for a STAT_ACCEPTED.

5 Revision information

Changes from previous released version to current revision v3-00	Page
Table 1: Startup sequence and shutdown sequence, information for powerup_select option after startup added.	5
Table 13: Start / Stop measurement, information for wake up device added.	14
Added a safety note for read frames under section 4.5	14

- Page and figure numbers for the previous version may differ from page and figure numbers in the current revision.
- Correction of typographical errors is not explicitly mentioned.

ABOUT ams OSRAM Group (SIX: AMS)

The ams OSRAM Group (SIX: AMS) is a global leader in intelligent sensors and emitters. By adding intelligence to light and passion to innovation, we enrich people's lives. With over 110 years of combined history, our core is defined by imagination, deep engineering expertise and the ability to provide global industrial capacity in sensor and light technologies. Our around 20,000 employees worldwide focus on innovation across sensing, illumination and visualization to make journeys safer, medical diagnosis more accurate and daily moments in communication a richer experience. Headquartered in Premstaetten/Graz (Austria) with a co-headquarters in Munich (Germany), the group achieved EUR 3.6 billion revenues in 2023. Find out more about us on https://ams-osram.com

For further information on our products please see the Product Selector and scan this QR Code.

Published by ams-OSRAM AG
Tobelbader Strasse 30, 8141 Premstaetten, Austria
ams-osram.com © All Rights Reserved.

Published by ams-OSRAM AG
Tobelbader Strasse 30,
8141 Premstaetten Austria
Phone +43 3136 500-0
ams-osram.com